



Fichas Técnicas: Estructuras de Datos Arbóreas

Referencia rápida para desarrolladores — BST · AVL · Árbol B/B+ · Trie · Heap

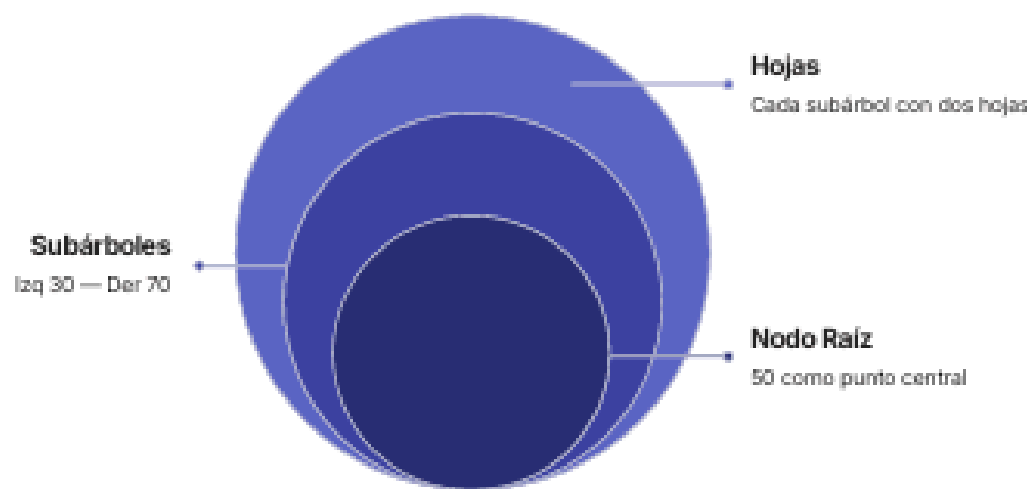
ESTRUCTURAS DE DATOS

COMPLEJIDAD BIG O

REFERENCIA TÉCNICA

BST — Árbol Binario de Búsqueda

Árbol donde cada nodo cumple: **hijo izquierdo < nodo < hijo derecho**. Permite búsqueda eficiente mediante división recursiva del espacio de claves.



Complejidad Big O

Operación	Promedio	Peor caso
Búsqueda	$O(\log n)$	$O(n)$
Inserción	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$

⚠ El peor caso $O(n)$ ocurre cuando el árbol está completamente desbalanceado (insertando en orden ascendente/descendente).

Caso real

Usado en **índices de bases de datos** simples y en implementaciones de conjuntos ordenados (`std::set` en C++, `TreeMap` en Java).

✓ Ventajas

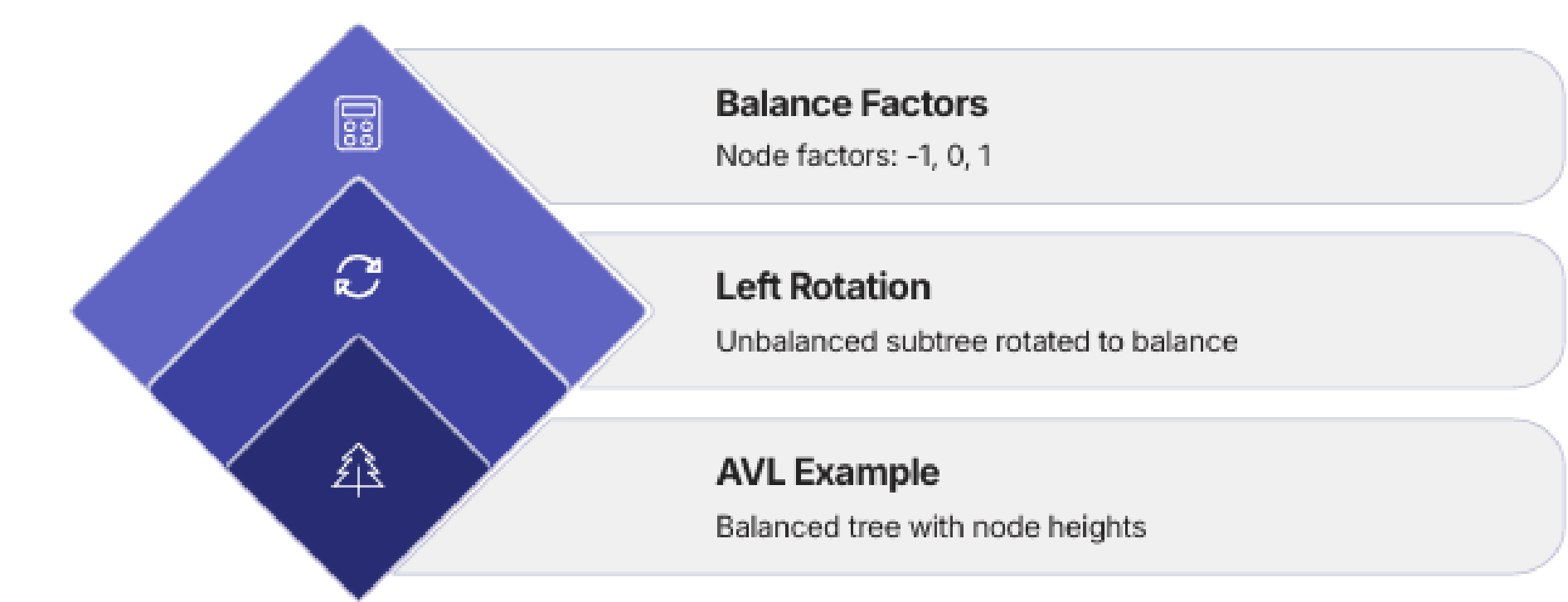
- Búsqueda rápida en árbol balanceado
- Recorrido in-order produce valores ordenados
- Implementación simple

⚠ Limitaciones

- Sin balanceo automático
- Degeneración a lista enlazada
- No apto para datos secuenciales sin rebalanceo

AVL — Árbol Balanceado por Altura

Extensión del BST con **auto-balanceo garantizado**. Mantiene el factor de balance (altura izquierda – altura derecha) en {-1, 0, 1} mediante rotaciones simples y dobles tras cada inserción o eliminación.



La ilustración muestra un árbol AVL balanceado, una rotación simple para corregir desbalance y la idea del factor de balance que se mantiene dentro de los límites permitidos.

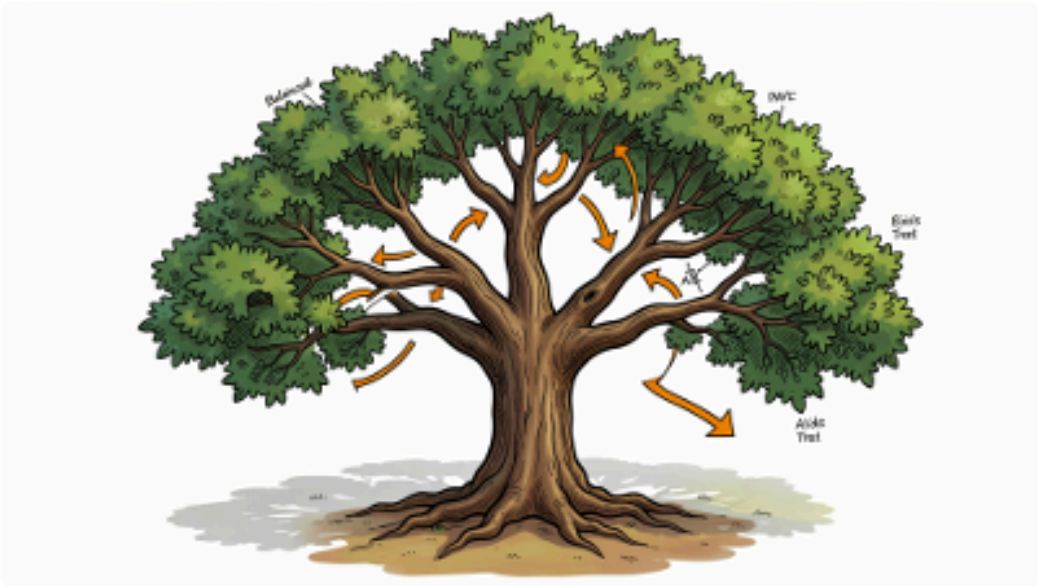
Complejidad Big O

Operación	Promedio	Peor caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

✓ A diferencia del BST, el peor caso está **garantizado en $O(\log n)$** gracias al balanceo automático.

Cómo funciona internamente

Tras cada operación, el árbol verifica los factores de balance desde el nodo afectado hacia la raíz. Si detecta desbalance, ejecuta **rotaciones (LL, RR, LR, RL)** para restaurar el equilibrio.



✓ **Ventajas**

- Búsquedas siempre $O(\log n)$
- Ideal para lecturas frecuentes
- Altura mínima garantizada

⚠ **Limitaciones**

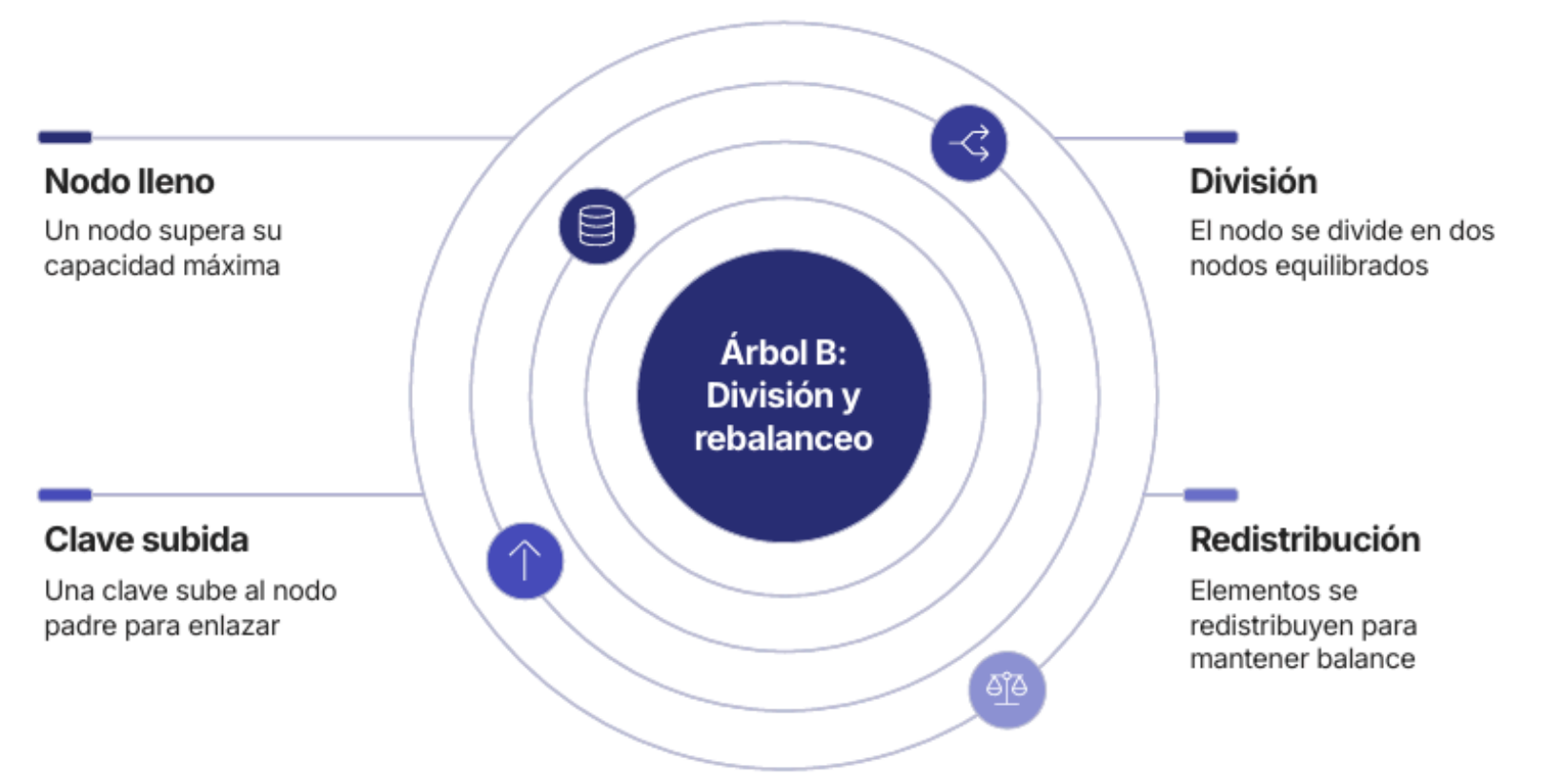
- Rotaciones costosas en escrituras masivas
- Mayor overhead de memoria (almacena factor de balance)

🏢 **Caso real**

Núcleo de **bases de datos en memoria** como Redis (Sorted Sets) y sistemas donde las búsquedas dominan sobre las escrituras.

Árbol B / B+

Árboles **multicamino autobalanceados** diseñados para minimizar accesos a disco. Cada nodo puede tener m hijos (orden del árbol). El B+ almacena todos los datos en hojas enlazadas, optimizando recorridos secuenciales.



Cuando un nodo excede su capacidad, el árbol realiza split y redistribución automática para conservar el balance.

Complejidad Big O

Operación	Promedio	Peor caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Con n claves y orden m , la altura es $O(\log_m n)$, lo que reduce drásticamente los accesos a disco respecto a un BST.

Ventaja clave: menos niveles, menos lecturas de disco y mejor rendimiento en bases de datos e índices.

Árbol B

Datos en nodos internos y hojas. Búsqueda puede terminar antes de llegar a las hojas.

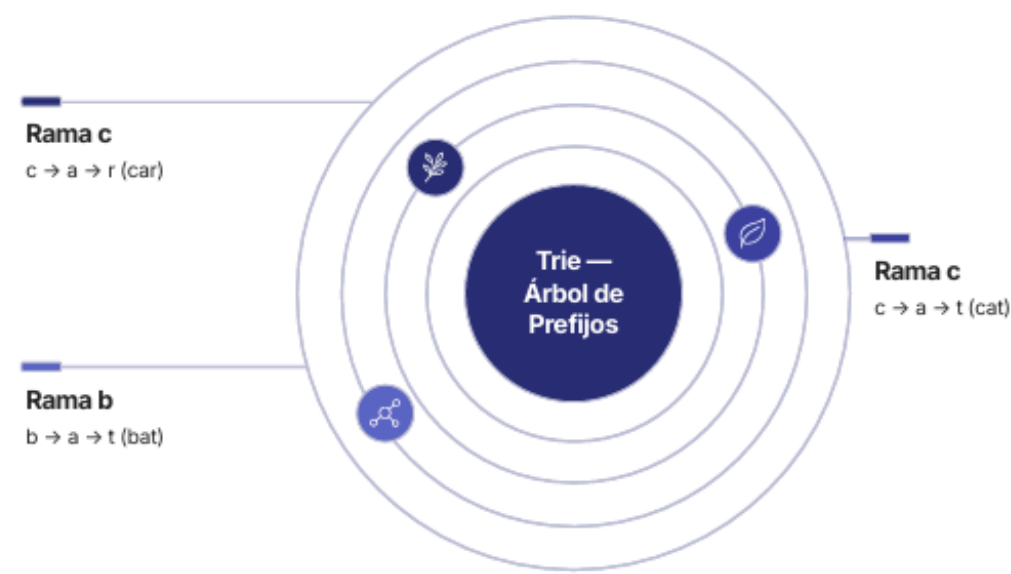
Árbol B+

Solo claves en nodos internos. Datos únicamente en hojas enlazadas. Óptimo para rangos.

MySQL InnoDB, PostgreSQL y casi todos los motores de bases de datos relacionales usan B+ como estructura de índice primaria.

Trie — Árbol de Prefijos

Estructura de árbol donde **cada nodo representa un carácter**. Las claves (generalmente cadenas) se almacenan como caminos desde la raíz. La profundidad es proporcional a la longitud de la clave, no al número de claves.



Ejemplo: las palabras *car*, *cat* y *bat* comparten prefijos, reduciendo espacio redundante.

Complejidad Big O

Operación	Complejidad
Búsqueda	O(L)
Inserción	O(L)
Eliminación	O(L)

L = longitud de la clave. Independiente del número total de claves almacenadas.

✓

Ventajas

- Búsqueda de prefijos en O(L)
- Autocompletado eficiente
- Sin colisiones de hash

⚠

Limitaciones

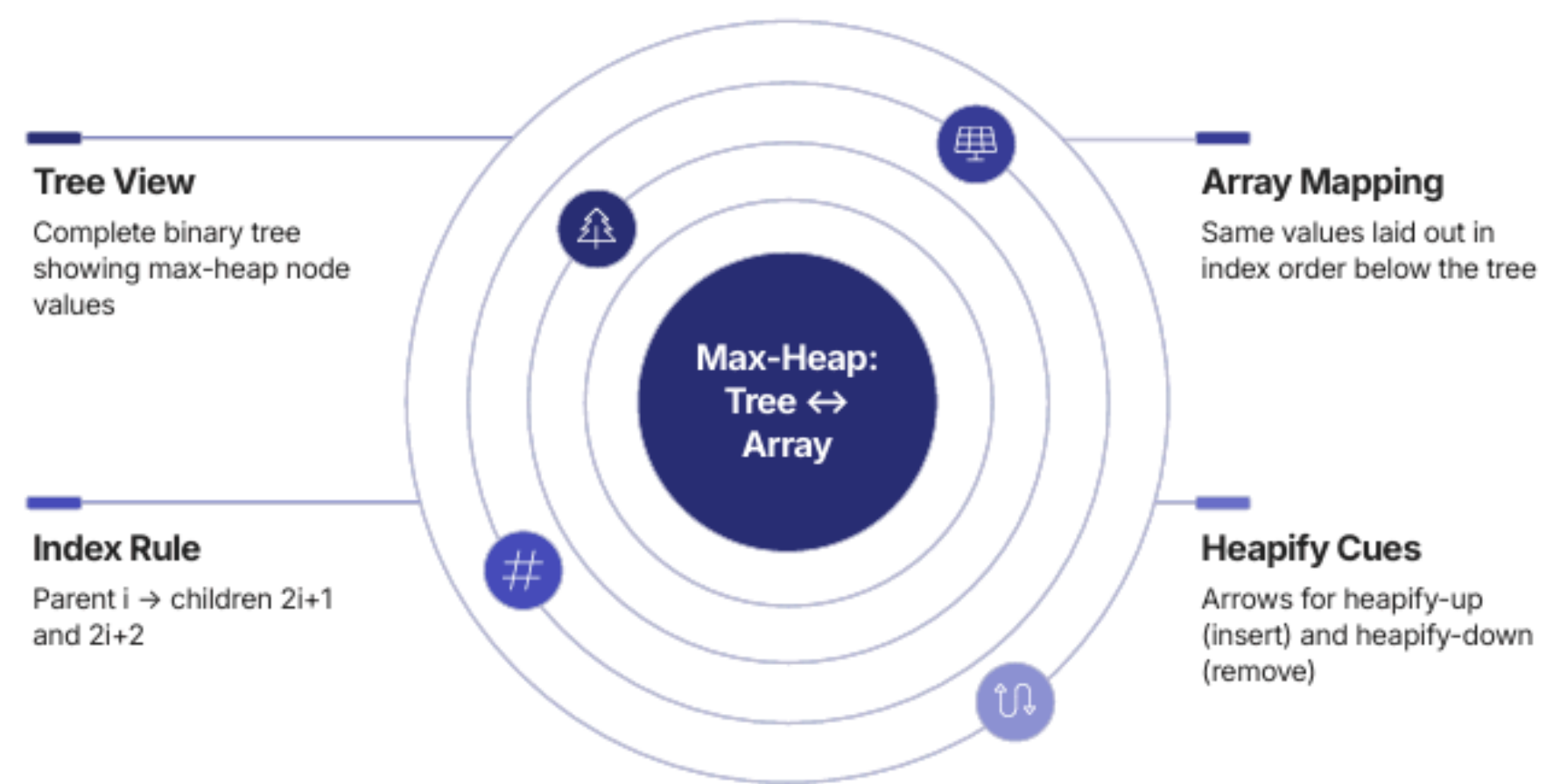
- Alto consumo de memoria
- No eficiente para claves numéricas largas

i

Motores de búsqueda, correctores ortográficos y autocompletado (Google Search, IDEs) usan Trie internamente.

Heap — Min-Heap y Max-Heap

Árbol binario **completo** que satisface la propiedad de montículo: en un **Min-Heap** el padre siempre es $\leq$ sus hijos; en un **Max-Heap** el padre siempre es $\geq$ sus hijos. Generalmente implementado sobre un array.



La estructura del heap permite acceder rápidamente al elemento prioritario y mantener la propiedad de montículo con operaciones locales de reajuste.

Representación y relación índice-nodo

Max-Heap de ejemplo: el mayor valor está en la raíz y cada padre es mayor o igual que sus hijos.

Índice	Nodo	Hijo izq.	Hijo der.
i	$arr[i]$	$2i+1$	$2i+2$

Esto hace que la navegación entre padres e hijos sea directa sobre el array, sin punteros adicionales.

Heapify

heapify-up: al insertar, el nuevo elemento sube mientras sea mayor que su padre.
heapify-down: al eliminar la raíz, el último elemento baja intercambiándose con el hijo prioritario hasta restaurar el orden.

✓ Ventajas

- Acceso $O(1)$ al mínimo o máximo
- Heapsort en $O(n \log n)$
- Eficiente en memoria (array)

⚠ Limitaciones

- No soporta búsqueda arbitraria eficiente
- No mantiene orden total de los elementos

Complejidad Big O

Operación	Complejidad
Acceso al mín/máx	$O(1)$
Inserción	$O(\log n)$
Eliminación (raíz)	$O(\log n)$
Construcción	$O(n)$

✔ Acceso al elemento prioritario es **$O(1)$** — la operación más frecuente en colas de prioridad.

🏠 Caso real

Dijkstra, A*, schedulers de SO y sistemas de eventos usan Heap como cola de prioridad central.

Comparativa General — Big O

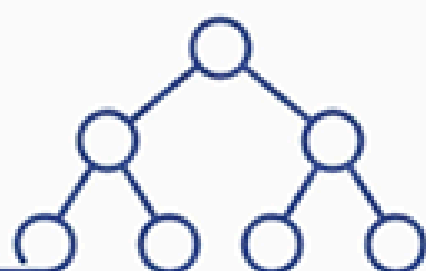
Resumen de complejidades en el **caso promedio** para las cinco estructuras. Úsalo como referencia rápida al elegir la estructura correcta para tu problema.

Estructura	Búsqueda	Inserción	Eliminación	Peor caso	Uso principal
BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Índices simples
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Búsquedas frecuentes
Árbol B/B+	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Bases de datos / disco
Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$	Cadenas / prefijos
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Cola de prioridad

📌 **Nota:** L = longitud de la clave (Trie). La "búsqueda" en Heap es $O(n)$ porque no garantiza orden lateral; su fortaleza es el acceso al extremo en $O(1)$.

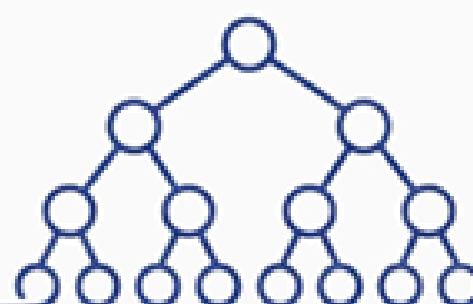
¿Cuándo usar cada estructura?

La elección depende del patrón de acceso, el tipo de clave y las restricciones de memoria o disco.



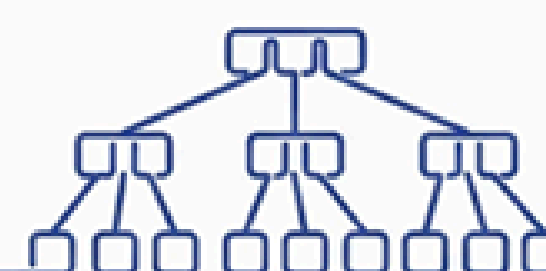
Árbol Binario de Búsqueda (BST)

Datos simples ordenados.
Sin balance estricto.
Uso educativo.



Árbol AVL

Búsquedas intensivas.
Búsqueda $\log n$
garantizada siempre.



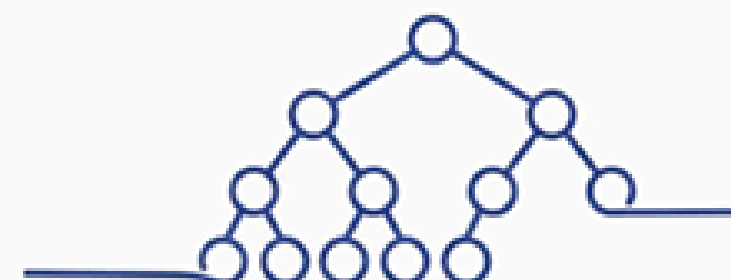
Árbol B / B+

Almacenamiento en disco. Grandes conjuntos de datos.
Índices de base de datos.



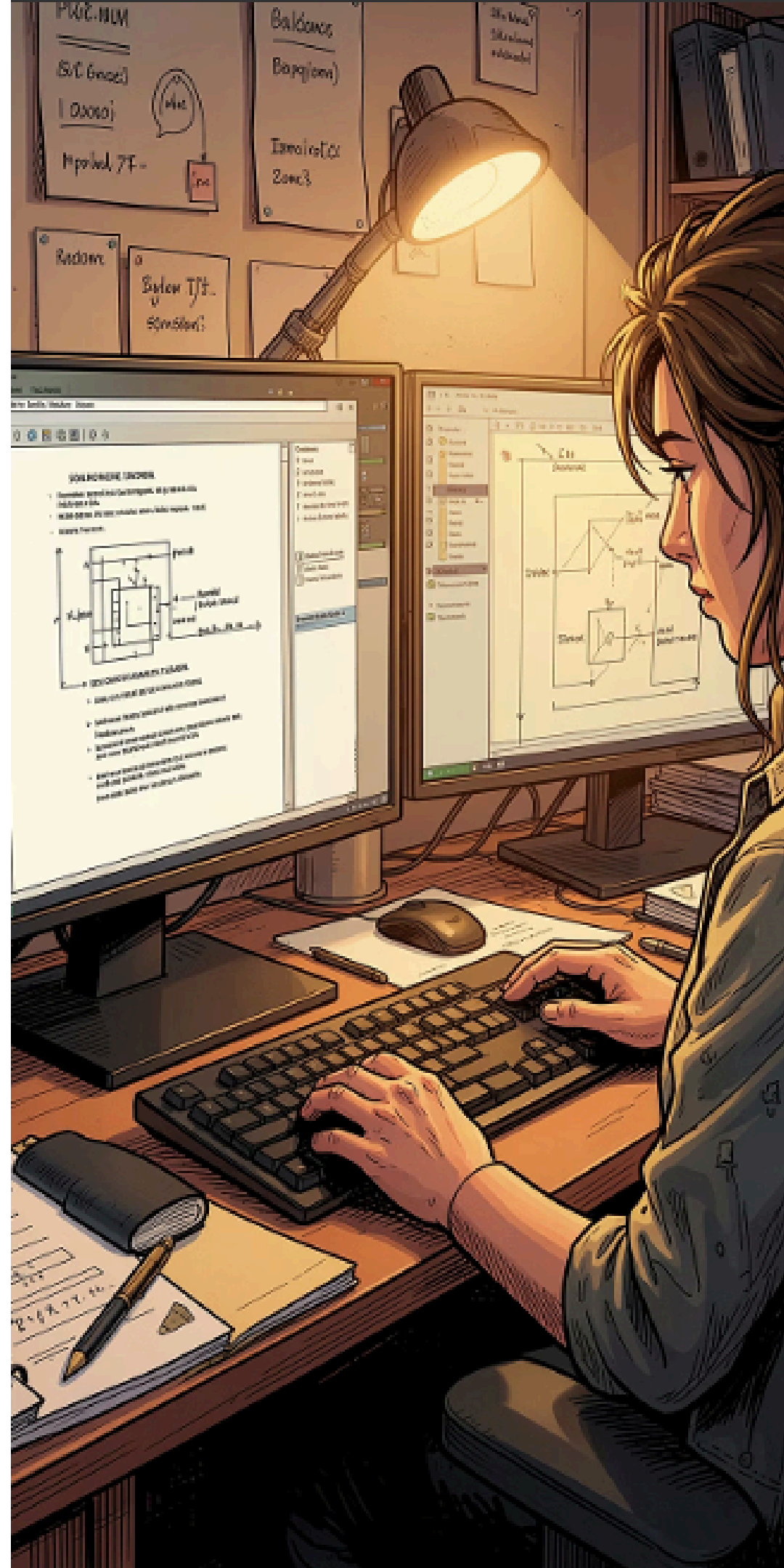
Árbol Trie (Clave)

Casmena. Búsqueda de prefijos.
Autocompletado y correctores correctores ortográficos.



Árbol Heap

Colas de prioridad.
Planificación. Algoritmos de grafos como Dijkstra.



Buenas Prácticas al Documentar Estructuras

01

Define el caso de uso primero

Antes de elegir la estructura, identifica si el cuello de botella es búsqueda, inserción, rango o prioridad.

02

Documenta el peor caso siempre

El promedio engaña en producción. El peor caso determina garantías de latencia en sistemas críticos.

03

Incluye un diagrama visual

Un árbol bien dibujado comunica en segundos lo que un párrafo tarda minutos en transmitir.

04

Ancla con un caso industrial real

Conectar la teoría con uso en MySQL, Redis o un sistema operativo hace la ficha técnica inmediatamente aplicable.

Resumen Final

Cinco estructuras, cinco casos de uso distintos. Conocer sus garantías teóricas es lo que separa una decisión de arquitectura sólida de una improvisada.

01
10

BST

Base de todo. Simple pero vulnerable al desbalance.



Trie

Maestro de cadenas y búsqueda de prefijos.



AVL

Balanceo garantizado. Ideal para búsquedas intensivas.



Heap

La cola de prioridad más eficiente para el extremo.



B / B+

El estándar de bases de datos y sistemas de archivos.